

Exploiting Similarity Opportunities of Emerging Vision AI Models on Hybrid Bonding Architecture

Zhiheng Yue¹, Huizheng Wang¹, Jiahao Fang¹, Jinyi Deng¹, Guangyang Lu¹, Fengbin Tu², Ruiqi Guo¹, Yuxuan Li¹, Yubin Qin¹, Yang Wang¹, Chao Li³, Huiming Han¹, Shaojun Wei¹, Yang Hu¹, Shouyi Yin¹

¹Tsinghua University, ²The Hong Kong University of Science and Technology, ³Shanghai Jiao Tong University

Abstract—While extensive research has focused on optimizing performance and efficiency in vision-based AI accelerators, an unexplored phenomenon, Clustering Similarity Effect, presents a significant opportunity for further improvement. This effect reveals that clusters of neighboring data points exhibit similar values, enabling the potential to skip redundant computations.

To fully capitalize on the potential of the Clustering Similarity Effect (CSE), this work integrates hybrid bonding DRAM technology. We conduct a comprehensive analysis of the associated design considerations and integration overhead. Leveraging these insights, we propose a novel CSE-aware architecture specifically tailored for hybrid bonding memory. This architecture facilitates similarity detection and adapts to the inherent data characteristics associated with CSE.

Compared with state-of-the-art 2D/2.5D AI accelerators, the hybrid bonding baseline demonstrates an average energy efficiency improvement of $2.89\times\sim 14.28\times$ and an area efficiency improvement of $2.67\times\sim 7.68\times$. Incorporating the similarity optimizations further enhances energy efficiency and area efficiency improvement to $5.69\times\sim 28.13\times$ and $3.82\times\sim 10.98\times$, respectively.

I. INTRODUCTION

The recent surge of vision-based AI has driven its adoption across diverse applications, from autonomous vehicles to facial recognition systems [1], [2], [3]–[7]. This widespread adoption has necessitated the development of more efficient computing solutions, specifically tailored to handle complex and data-intensive tasks. Consequently, there has been a significant rise in the design and development of hardware accelerators [8]–[14]. However, amidst these advancements, there lies an often-overlooked phenomenon that has yet to be fully exploited.

This work delves into representative vision-based AI workloads, identifying the presence of the *Clustering Similarity Effect* (CSE) across memory devices. This effect reveals a pattern in data representation and storage, observed both within input activations and the underlying physical memory. Specifically, it describes that within an input image, clusters of neighboring data points that share contextual similarities often exhibit similar values. The smaller the size of the cluster, the higher the data similarity.

Intriguingly, this effect mirrors a pattern observed in physical memory devices. We find that *smaller* clusters of neighboring memory cells tend to contain *more similar* values as well. Therefore, the absolute input can be substituted with the difference between similar data, which is expected to be sparser and could be accelerated by sparse engines. It indicates that the *finer-grained the memory blocks are clustered for*

data storage, the greater the degree of data sparsity that can be exploited.

However, traditional 2D/2.5D DRAM architectures present limitations in exploiting the Clustering Similarity Effect. These limitations stem from restricted parallel similarity detection capabilities due to limited I/O parallelism and the inherent data dispersion caused by memory interleaving, which directly contradicts the principle of CSE. These constraints motivate the exploration of highly parallel, non-interleaving memory architectures like hybrid bonding DRAM [15], [16]. This technique integrates the memory and the computing unit more closely through numerous metal pillars. Each logic block can directly access its corresponding fine-grained memory block (e.g., 32MB memory bank size) via uniformly distributed copper pillars and dedicated memory controllers. This advanced memory technology not only aligns more closely with CSE by facilitating **tighter clustering** of memory blocks without interleaving but also leverages the **inherent parallelism** among multiple memory banks through high bandwidth, thereby maximizing the benefits of CSE in vision-based workloads.

Despite the promising attributes of hybrid bonding integration DRAM, directly applying it poses several notable challenges. Firstly, the process of detecting similar data within a DRAM bank, which requires an exhaustive search, can negate the potential benefits brought by the CSE. Secondly, most of sparse engines do not adequately consider the type of sparsity induced by data similarity, hindering their ability to fully exploit these data characteristics for computational acceleration. Thirdly, achieving workload balance becomes critical when data similarity varies across DRAM banks. However, inter-bank data movement for balancing introduces significant traffic congestion and increased power consumption, ultimately degrading the overall efficiency.

To fully harness the potential of the Clustering Similarity Effect (CSE) with hybrid bonding DRAM, this work conducts a comprehensive analysis of design considerations of hybrid bonding architectures. Leveraging these insights, we propose a novel CSE-aware architecture specifically tailored to this advanced memory technology. This architecture captures the inherent data sparsity associated with CSE, enabling significant acceleration in the processing of vision-based AI models.

Fig. 1 showcases the practical implementation. This architecture introduces a novel SRAM-based Content Addressable Memory (CAM) to exploit data similarity. It efficiently locates suitable hotspots – represents clustering data similarities. The

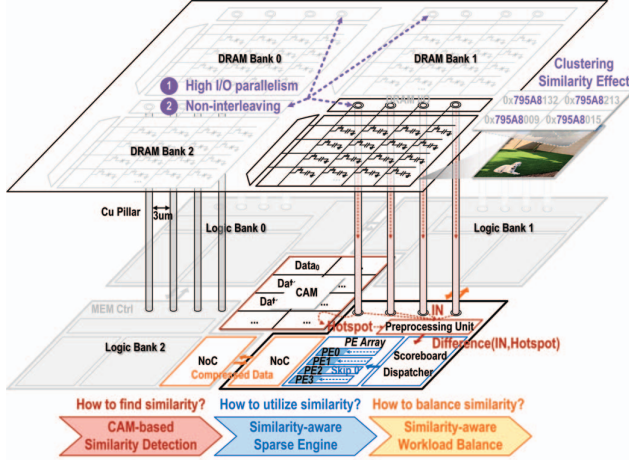


Fig. 1. Hybrid bonding DRAM scheme and the overall architecture.

pre-processing unit computes the sparse differences between them and subsequent similar inputs. The architecture incorporates an optimized Processing Element (PE), which efficiently allocates resources based on the sparsity patterns. Additionally, to ensure bank-level workload balance, we implemented a similarity-based workload compression mechanism. This mechanism dynamically redistributes workloads, preventing traffic congestion and excessive power consumption.

To the best of our knowledge, this is the first work that thoroughly investigates the integration of hybrid bonding technology for emerging vision models while exploiting the opportunities presented by clustering data similarity.

- We analyze the data similarity of AI tasks, design and evaluate the similarity metric in different networks and regions, and summarize the clustering similarity effect.
- We conduct a comprehensive analysis of hybrid bonding DRAM, exploring its potential to fully exploit CSE opportunities. This involves analyzing design considerations and accommodating integration overhead, ultimately implementing a hybrid bonding architecture.
- We introduced a novel hardware design specifically tailored for hybrid bonding memory. This design enables highly efficient similarity detection and adapts to the inherent data characteristics associated with CSE, significantly reducing redundant computations.
- We build a prototype and verify its effectiveness with an augmented simulator. The 17nm hybrid bonding DRAM lib is provided by the vendor, the logic die is implemented and synthesized in 12nm technology. The systematic vision benchmark suites are evaluated.

II. DESIGN CONSIDERATIONS OF HYBRID BONDING

This work leverages a 3D hybrid bonding architecture to accelerate the execution of vision AI models. This approach vertically stacks a DRAM die on top of a logic die. Interconnection between these dies is achieved through copper pillars, serving as both communication channels and power delivery pathways [16], [17]. In comparison to traditional 2D/2.5D

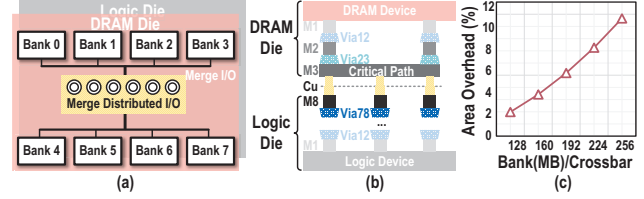


Fig. 2. Data space extension overhead. (a) Centralized controller scheme. (b) Cross-section view of the controller. (c) Crossbar scheme.

architectures, the 3D DRAM exhibits superior bandwidth and enhanced I/O parallelism, characteristics that align synergistically with the objective of exploiting CSE opportunities. However, implementing this architecture requires consideration of various design aspects. This section details the design considerations associated with the hybrid bonding architecture.

A. Higher I/O Data Rate or Higher Parallelism

Optimizing DRAM bandwidth involves a trade-off between I/O data rate and parallelism. Higher data rates achieved through increased frequencies require complex analog-based Physical Layers (PHYs) for synchronizing read/write timings. This complexity incurs significant overhead and restricts the achievable number of I/Os. 3D hybrid bonding architectures offer an alternative, enabling exceptionally high I/O parallelism (up to 110,000/mm² with a 3 μ m pitch) due to dense integration. Leveraging this density, we propose a low-frequency I/O design where DRAM bank data directly interfaces with logic units via copper connections, eliminating the need for asynchronous PHYs. This direct connection significantly reduces parasitic resistance ($<0.5 \Omega$), leading to improved energy efficiency in memory access (0.88 pJ/bit). Moreover, hybrid bonding directly transfers data to logic units from bank-level I/O, bypassing the need for multi-stage arbiters.

B. Distributed or Unified Controller

Hybrid bonding DRAM offers high bandwidth through parallel banking structures. However, individual I/O interfaces can only access their corresponding private DRAM bank. To consolidate distributed banks, a centralized DRAM controller aggregates I/O from different banks (Fig. 2(a)(b)). However, due to the limited 3 metal layers in DRAM compared to logic technology, long routing within DRAM incurs a significant performance degradation (22%) and area overhead (80%). An alternative crossbar-based approach within the logic die is implemented and synthesized in 12 nm CMOS technology. The results in Fig. 2(c) reveal that the area cost reaches 6.2% and 10.6% of the logic Die for merging 192MB and 256MB DRAM storage, respectively. Consequently, existing methods for consolidating banks are not efficient due to hardware costs. Therefore, to effectively leverage the bandwidth benefits of hybrid bonding DRAM, the accelerator must be designed to accommodate and exploit this distributed characteristic.

C. Heat Dissipation

Our work tackled overheating in the chip by quantitatively analyzing thermal resistance (R_{θ}), representing heat flow

resistance from the core to the environment. Initially, R_{Θ} was 4.214°C/W, a wind cooling system drastically reduced it to 0.10°C/W in thermal field experiment. The cooling technique is effective because the DRAM die is positioned directly on top of the chip, allowing for close contact with the cooling system. While the logic die is sandwiched between the package board and DRAM die, it is bonded to the DRAM via copper pillars, whose excellent heat conductivity allows the heat source to directly access the cooling system. And logic die is thinned to $\sim 10\mu\text{m}$ before integration to reduce heat accumulation.

When encountering increased heat, DRAM cells experience higher leakage current, necessitating more frequent refreshes to maintain data integrity. To address this, the design dynamically adjusts the refresh period based on temperature. Below 85°C, refreshes occur every 32ms, utilizing 0.6% of bandwidth. Exceeding 85°C triggers a switch to 16ms refreshes, modestly increasing bandwidth usage to 1.2%. The logic die also employs dynamic voltage/frequency scaling (DVFS). DVFS reduces the operating frequency of components on the critical path. Like the content addressable memory increases sense time from 0.5449 ns at 25°C to 0.8752 ns at 85°C.

D. Reliability

Compared to conventional 2D/2.5D integration, the 3D hybrid bonding architecture induces additional Cu pillar, through-silicon vias (TSVs), distributed DRAM controllers. Corresponding reliability issues are analyzed: (1) The maximum resistance shift of Cu bonding is limited to -6.95%. Cu pillar achieves 99.999% reliability with additional backups for critical functionalities, (2) TSV has a larger size than Cu pillar (25 μm vs. 3 μm) with multiple backups per connection mitigating potential errors, and (3) The low-frequency data I/O enables a simple and error-resilient PHY-free DRAM controller. The extensive BIST/redundancy repair mechanisms result in a 99.872% yield of good DRAM banks per chip.

E. Integration Overhead

Each bank of 3D DRAM requires its own dedicated memory controller, leading to a significant number of controllers. Each controller supports address decode, timing control, and BIST, occupies approximately 17% of the logic bank area.

Although TSVs are no longer used for communication paths like HBM/HMC architectures, TSVs are still utilized for off-chip I/O and power delivery, necessitating their penetration through the logic die (25 μm^2 each). They occupy space and prohibit routing or placement of logic devices. To meet the power requirements of chip, the design utilizes a pitch of 79.6-87.18 μm for their on-chip distribution (each TSV supports 16 $\sim$ 36 mA). TSV introduces 3 mm² area overhead in this work.

III. OPPORTUNITIES AND CHALLENGES

This section conducts a comprehensive analysis of data similarity using state-of-the-art vision-based AI workloads. The experiments uncover the latency/power-saving opportunities brought by clustering similarity effect (CSE). Our characterizations reveal that 3D DRAM architecture is inherently well-suited to exploit CSE benefits, unlike other DRAM types.

We also identify challenges associated with exploiting CSE opportunities within hybrid bonding architectures.

A. Opportunities: Clustering Similarity Effect

Vision AI tasks present that neighboring activations within a bounding region exhibit similarity, as reflected by the low Euclidean distance between data elements. Eq. 1 formalizes this data similarity metric, where $\mathbf{X}$ and $\mathbf{Y}$ denote two elements. Data pairs exceeding a 0.8 threshold are considered ‘similar’ and the similarity ratio for a region quantifies the proportion of such pairs within the total number of examined pairs. The similarity originates from two aspects. First, neighboring activations exhibit inherent similarity due to the continuous contextual information, leading to similar features extracted by subsequent network layers. Second, substantial data exhibit relatively small values after layers of processing, which increases the likelihood of similarity.

$$\text{Similarity} = \frac{1}{1 + \sqrt{(\frac{1}{n} \times \sum_{i=1}^n (X_i - Y_i)^2)}}. \quad (1)$$

To validate the existence of similarity, the data characteristic after the processing of the Vision Transformer Base (ViT-B) [6], ResNet-50 [18], Stable Diffusion [19] based on ImageNet dataset [20] is analyzed. Fig. 3(a) shows the similarity ratio of each 32 $\times$ 32 region in ViT-B. The average similarity ratio reaches 81.6% in the first layer and remains above 61.9% even after processing through multiple layers. Furthermore, Fig. 3(b) reveals that the similarity effect is even more pronounced in smaller 16 $\times$ 16 regions, with an average ratio of 81.9% in the first layer and at least 69.9% after the encoder layer. As illustrated in Fig. 3(c)3(d), the similarity characteristic also exists in the convolution-based network ResNet-50. Both the 7 $\times$ 7 and 4 $\times$ 4 regions achieve an average similarity ratio exceeding 60.9% and 64.6%, respectively. Fig. 3(f)3(e) illustrate the similarity results of Stable Diffusion model, the average similarity ratio reaches 46.7% for 6 $\times$ 6 patches and 52.5% for 3 $\times$ 3 patches.

Fig. 4 visualizes the relationship between similarity ratio and region size. In ViT-Base benchmarks, shrinking the region size from 64 to 16 leads to an average increase of 17.7% in the similarity ratio. Similarly, ResNet-50 exhibits an average increase of 15.2% in similarity ratio when shrinking region size from 10 to 3. And the distribution differentiation (Max - Min) also demonstrates a reduction of 1.82 $\times$. This trend extends to the Stable Diffusion model, which achieves an 18.8% enhancement in similarity by decreasing the region size from 10 to 3. The results can be explained by data continuity. Within a smaller region, if two data points undergo continuous changes [21], they are more likely to contain continuous context and higher similarity ratios. Therefore, a cluster of neighboring stored data tends to contain similar values. We conclude this phenomenon as ‘**clustering similarity effect**’.

B. Exploiting CSE: 3D DRAM vs. 2D/2.5D DRAM

Leveraging CSE, we can transform highly similar data into sparse representations, capturing sparse differences and elimi-

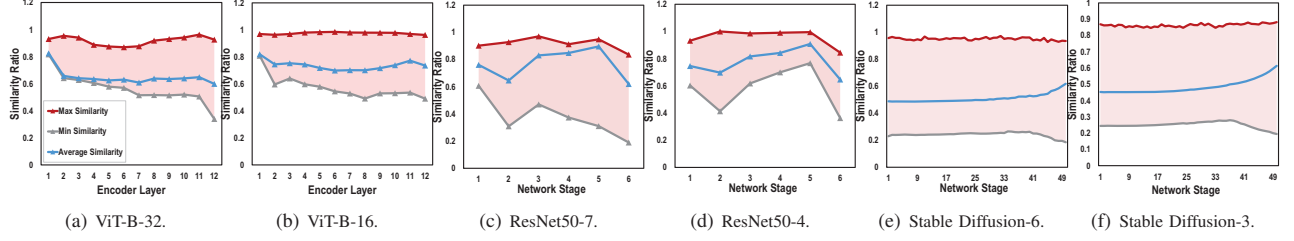


Fig. 3. Similarity experiment of Vision Transformer, ResNet-50 and Stable diffusion-V1. The entire Vision Transformer is divided into 12 encoder layers, and ResNet-50 is divided into 2 FC and 4 convolution stages. Stable diffusion adopts 860M UNet and CLIP ViT-L/14, and PLMS sampling algorithm.

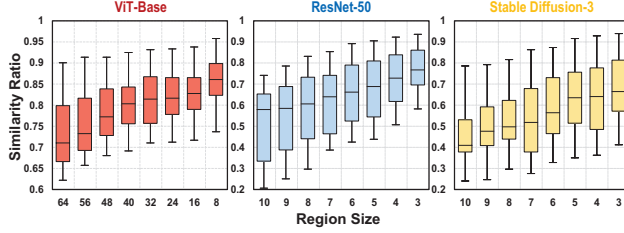


Fig. 4. Clustering similarity effect versus region size.

nating redundant computations. 3D DRAM's unique hardware characteristics inherently align with CSE principles. The synergy between 3D DRAM and CSE achieves performance gains unattainable with 2D/2.5D DRAM. The limitations of 2D/2.5D DRAM in exploiting CSE are detailed below.

1) *Limited I/O parallelism*: Sec. II-A highlights the limited I/O parallelism in conventional DRAM (HBM 1024, DDR 64). This inherently restricts parallel exploitation of the CSE. Additionally, larger data spaces per I/O channel, contradicting CSE principles, lower the similarity ratio within individual I/O streams. We conduct an experiment where identical benchmark workloads are allocated to a simulated DRAM with varying numbers of I/O channels. The average similarity ratio for each I/O is then evaluated, assuming data is stored in consecutive physical addresses. As depicted in Fig. 5, the 4×4 I/O exhibited the lowest similarity ratio of 30.6%, the 8×8 and 16×16 achieve higher ratios of 49.5% and 74.1%, respectively.

2) *Memory Interleaving*: Prior experiments assume consecutively addressed memory. However, conventional DRAM employs memory interleaving, distributing addresses across multiple banks for increased bandwidth. This allows simultaneous activation of row buffers, potentially hiding the access latency of individual banks. But interleaving disperses data, contradicting the CSE principle. Conversely, 3D DRAM already exhibits bank-level parallelism, negating the need for interleaving to enhance parallelism. As demonstrated in Fig. 6, the hybrid bonding approach achieves a 25.1% higher similarity ratio compared to a hash-interleaved DRAM.

3) *Low Memory Bandwidth*: CSE eliminates redundant computations but cannot fundamentally address memory access limitations for memory-bound tasks. Roofline models (Fig. 7) of 3D DRAM and 2D/2.5D DRAM reveal that attention-based, LSS, and Diffusion workloads suffer from low operational intensity, rendering them memory-bound.

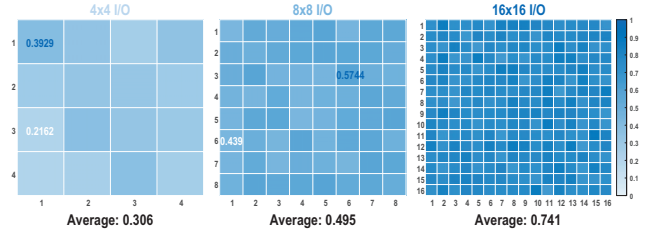


Fig. 5. Clustering similarity ratio versus I/O number.

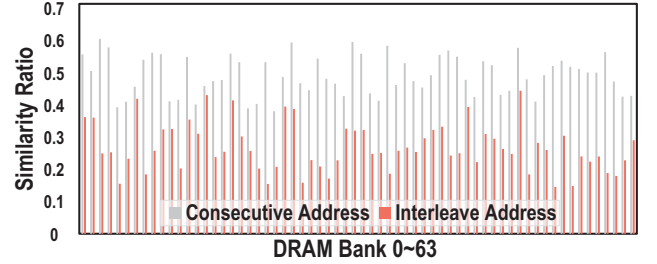


Fig. 6. Clustering similarity ratio versus interleaving.

Compared to 2D/2.5D DRAM, 3D DRAM offers a $19.2 \times$ bandwidth improvement, transforming most memory-bound workloads into compute-bound ones. However, the 3D integration incurs an area overhead that competes with the computing unit budget, resulting in a $1.27 \times$ lower throughput ceiling. Therefore, CSE presents a promising solution to further improve performance by effectively utilizing the computing unit.

In conclusion, by allocating workloads to individual banks without interleaving and boosting memory bandwidth via high I/O parallelism, hybrid bonding integration facilitates efficient parallel detection of similar data within each bank. Therefore, the hybrid bonding integration is leveraged to accelerate computing by fully exploiting the CSE opportunity.

C. Challenges of deploying 3D DRAM for CSE

However, the exploitation of data similarity in hybrid bonding architecture raises three pivotal challenges.

1) *Overwhelming Similarity Search*: The location of similar data varies depending on the application scenario. These similar data pairs can reside within adjacent columns or rows of a DRAM bank. To detect the similarity, an exhaustive iteration, as exemplified in Fig. 3, is impractical in hardware due to its time and energy consumption. Every data point would

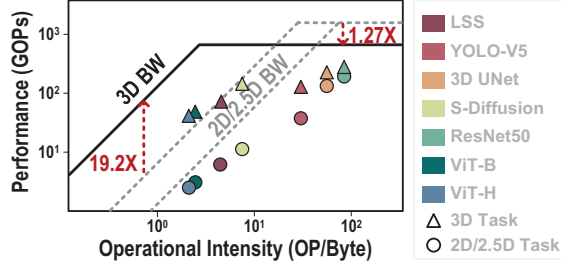


Fig. 7. Roofline model of 3D and 2D/2.5D DRAM architecture.

require multiple DRAM accesses to compare with all potential candidates, negating the potential benefits of exploiting CSE.

2) *Similarity-unaware Sparse Engine*: Many works have demonstrated the efficacy of reducing redundant computations caused by sparsity. The corresponding accelerator could handle different types of sparsity, ranging from bit-wise to structured tensor sparsity [22]–[25]. But most of the designs take advantage of intrinsic sparse features inherent in specific algorithms, e.g., ReLU [26], Pruning [27]. However, few accelerators pay attention to the sparsity caused by data similarity, hence cannot efficiently exploit data characteristics to accelerate computing.

3) *Bank-level Similarity Imbalance*: Each 3D DRAM bank is independently controlled and distributed workload. However, the software distributes workload based on data size or the number of operations and is unaware of the similarity beforehand, which will be distinguished by hardware at runtime. Each bank may encounter imbalanced similarity scenarios. Attempting to balance this workload through data movement between independent banks can lead to significant traffic congestion and increased power consumption.

To implement a CSE-aware 3D architecture, we manage to answer three questions. **How to find the similarity? How to utilize the similarity? How to balance the similarity?**

IV. SIMILARITY DETECTION

The first step is to identify similar data pairs. Accordingly, a content-addressable memory is proposed in this section, which can efficiently detect similar data in diverse applications.

A. Hotspot Content Addressable Memory

The core concept of the similarity detection scheme is to utilize the representative data, which is named *hotspot*, to find element-wise similarity without snooping for all DRAM data. As revealed in Fig. 8, Within a region, data points designated as hotspots exhibit characteristics indicating contextual alignment with their neighbors. This alignment is characterized by high similarity scores. Therefore, by collecting the hotspots from different regions in a buffer, we can obtain the similarity information with much lower overhead.

To exploit CSE during network processing, the first burst of readout data from the DRAM bank is compared with the hotspot buffer. Once the readout data matches with one of the entries, the matched hotspot becomes the reference for subsequent data. Then following readout data is reformulated into: hotspot + $\Delta(\text{hotspot}, \text{data})$. With data similarity, the

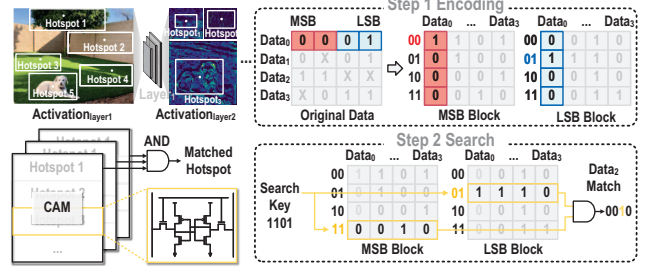


Fig. 8. Hotspot content addressable memory.

differential result is usually sparse and can be accelerated. To facilitate hotspot detection, a content-addressable memory (CAM) is employed. This CAM stores key hotspots and enables parallel comparisons within a single cycle.

SRAM-based Content Addressable Memory We propose an SRAM-based CAM to achieve fast in-memory data comparison. The CAM is based on compact six-transistor (6T) SRAM, and the search is realized by normal read and near memory AND operation. Fig. 8 details the operation of CAM.

The data is first encoded in 2-bit slices. Taking 4-bit encoding as an example, each two bits corresponds to four possible values and needs four entries to store the encoded data. When a certain entry address matches the logical value, its content is set as 1, and the others remain 0. As exemplified in Fig. 8, the values of the most significant two bits of data_0 are '00' so the entry at address '00' is written as 1, and 01/10/11 entries are set as 0. This CAM efficiently handles ternary data (0/1/X) by setting multiple entries to 1. For example, data 'X0' writes entries at both addresses '00' and '10' as 1.

The search operation involves reading the entry whose address aligns with the search key. Each bit in the read-out result indicates whether the data's 2-b fragment matches the search key. For instance, if the search key is '1101', the row at address '11' of the MSB block and '01' of the LSB block will be readout. Finally, these read-out bits are ANDed, meaning all 2-b fragments should match with the search key. The result is '0010', indicating that only data_2 matches with the search key. To store one ternary data (0/1/X), SRAM-based CAM takes merely 6 transistors/cell $\times$ 2 cells, while the conventional CAM requires at least 9~12 transistors/cell $\times$ 2 cells [28]–[32]. Therefore, this SRAM-based CAM outperforms in area efficiency. Moreover, the ability to split search keys into 2-bit slices allows for flexible bit-precision support.

B. CAM-based Similarity Detection

The hybrid bonding DRAM array includes a row buffer to latch data. The peripheral read out the row buffer in a burst-by-burst manner ($\times 64$ burst in this work). The first burst of DRAM data will check the CAM for matched entries. If a match is found, the matched hotspot is designated as the reference to generate differential results with the subsequent DRAM burst. Fig. 9 details similarity detection workflow.

Step 1 The DRAM controller activates the word line (WL) of the target row based on the provided physical address. Data

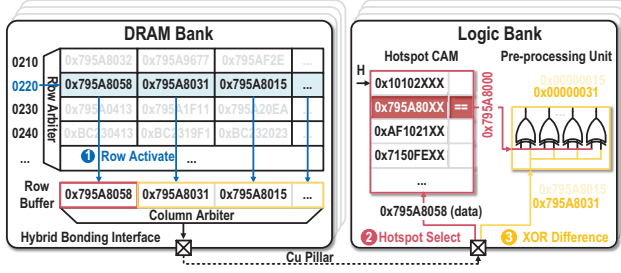


Fig. 9. Content addressable memory similarity detection workflow.

is read out through BL/BLB, then the sense amplifier pulls the voltage of BL/BLB to VDD/GND with an interleaved latch. The parallel latch forms the row buffer to temporally store the row data. The column arbiter selects data burst from the row buffer and transfers it through the Cu pillar to the logic die.

Step 2 The first data burst undergoes a comparison with a content-addressable memory (CAM) to identify a matching hotspot. When a matched entry exists, one of the tag bits equals 1. The corresponding hotspot becomes the reference point for subsequent data bursts within the same row. Thus, the CAM hotspot can represent regional information with authority. In the absence of a match, the first data burst itself serves as a temporary reference. The Sec. IV-C will introduce the details of CAM initialization and update.

Step 3 A near-memory pre-processing unit, utilizing parallel XOR gates, computes the difference between the identified reference and subsequent data bursts within the row. The data from hybrid bonding I/O is directly processed by the XOR gates. Assuming the similarity exists, the output data of the XOR unit is sparser in the bit-level (more bit 0). The subsequent sparse engine could take advantage of this feature to speed up computing.

To amortize the CAM area overhead and reduce routing complexity, a single CAM unit is shared by four neighboring DRAM banks. This potential bottleneck is addressed through a scheduling strategy that utilizes the CAM efficiently. Since only the first burst from each row requires comparison with the CAM to identify a reference hotspot, subsequent bursts within the same row can leverage this reference. Therefore, we employ an interleaving scheme for the first burst accesses from different banks, minimizing the likelihood of conflicting CAM access requests.

C. Hotspot Setup and Update Policy

In this subsection, the strategies for selecting and updating hotspots in CAM are introduced in detail.

Initialization: During initial data write operations, the NoC sends the off-chip data to the memory controller below the target bank position, and the hybrid bonding pillars deliver data to the desired DRAM banks. Since the data must travel to the logic unit, the pre-processing unit performs similarity comparison and identifies hotspot data before writing it into DRAM. Due to the DRAM I/O configuration, data writing also occurs in a burst manner. So the later burst cannot be

compared with earlier bursts that have already been written into DRAM. Only the first burst can compute similarity with the entire row data, which is first designated as the ‘candidate’.

Each hotspot should be unique and represent a region of similarity. Thus, the candidate is first compared with the existing CAM content. The candidate is discarded if it matches the current CAM entry. If not, the candidate is compared with subsequent bursts within the same row to assess its regional similarity. As previously mentioned, a lack of similarity does not hinder performance, negating the need for exhaustive comparisons within the row. If the candidate demonstrably exhibits similarity with subsequent bursts, it is written to the CAM. Depending on previous candidate test results, the specific number of least significant bits is set to state ‘X’, which improves the possibility of matching.

Evict Policy: When the CAM is full and the content search still misses, the readout data replaces the out-of-date CAM entries. The new candidate data also undergoes a test similar to the initialization. If proven effective, the data evicts existing entries in CAM.

We adopt the simplest *Oldest Address* (OA) strategy, in which the data associated with the oldest entry in the CAM is replaced with the new data. As Fig. 9 shows, a head pointer marks the oldest entry. When the new hotspot replaces the oldest one, the pointer moves to the adjacent entry. The simplest mechanism is adopted to maintain the CAM since a similar scenario is random and unpredictable. The sophisticated *Least Recently Used* (LRU) cache policy does not improve the hit rate but incurs 4.2% extra area overhead.

To summarize, an SRAM-based CAM stores hotspots that are similar to neighboring data. DRAM Data burst can efficiently select similar hotspots from CAM, and a pre-processing unit calculates the XOR difference between the matched hotspot and the readout data. Assuming similarity exists, the difference result becomes sparser and can be accelerated by the following sparse engine. Without compromising accuracy, the computation of input is reformulated to the processing of sparse differences between the hotspot and original input.

V. SIMILARITY COMPUTATION

Leveraging the sparsity generated by the similarity detection technique, which computes the difference between similar data elements, this work proposes a custom accelerator optimized for similarity-induced sparse computing.

A. Progressive Sparsity Detection (PSD)

The XOR operation between the data and a representative hotspot naturally exploits inherent similarities at the bit level. Specifically, the higher-order bits often exhibit sparsity, characterized by consecutive leading zeros. This characteristic sparsity motivates the design of our novel Progressive Sparsity Detection (PSD) technique, which efficiently identifies non-zero elements crucial for subsequent computations.

The PSD operates as Fig. 10 shows, the complete data is divided into multiple blocks and a scoreboard maintains the sparsity status of each block, indicating whether it contains

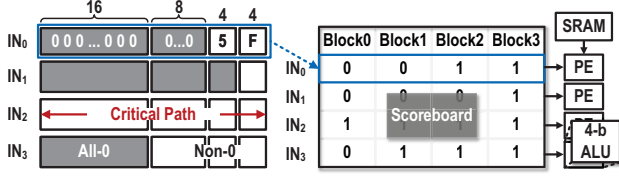


Fig. 10. The illustration for progressive sparsity detection.

all zeros or not. Specifically, each data is first split into two halves, and the scoreboard prioritizes checking the higher-order blocks for zeros due to the tendency for sparsity to emerge in those regions. If a block is found to be non-zero, it initiates computation for that block. Otherwise, it undergoes further division into halves, and the process repeats until the blocks reach a granularity of 4 bits. This selection of a 4-bit granularity is informed by bit-level similarity analysis, which indicates that only 21.3% of 4-bit data pairs exhibit similarity. PSD effectively exploits the sparsity induced by similarity, leading to significant computational acceleration.

Fig. 10 demonstrates the computing process, in which parallel PE units consume input data from DRAM and weights from SRAM. The scoreboard issues parallel non-zero blocks starting from the least significant 4-bit and the all-0 data block is skipped. Each cycle the PE completes 4-bit operations. However, data similarity can vary significantly across different input data points. This results in uneven sparsity, where individual data exhibits diverse sparsity patterns. This imbalance in sparsity poses a challenge and limits the performance on the critical path. This highlights the need for further advancements in managing sparsity imbalance to fully unlock the potential of similarity-induced sparse computing.

B. Progressive Sparsity Balance (PSB)

To alleviate the inter-PE sparsity imbalance, the scoreboard and dispatcher reallocate workload based on the sparsity state. The balancing procedure is detailed in Fig. 11.

The PE unit operates with 4-bit granularity. Therefore, the two rightmost data blocks ([7:4] and [3:0]) can be processed within a single cycle, eliminating any performance impact from workload imbalance. The scoreboard will issue parallel input data to PEs as soon as possible. For 8-bit data blocks, requiring two cycles for completion, the dispatcher optimizes workload distribution. It identifies non-zero elements in the scoreboard entries, allowing each PE with non-zero input to share the workload with an idle PE. For example, the 8-bit IN_2 is split and shared between PE0 and PE2. Likewise, IN_3 is allocated to PE1 and PE3. For the 16-bit block, IN_2 contains a non-zero 16-bit block and the dispatcher notices 3 idle units according to the scoreboard. Therefore, PE0, PE1 and PE3 each receive a 4-bit slice of IN_2 , while PE2 completes the remaining 4-bit computing. With workload balance, the computing period reduces from 8 cycles to 4 cycles.

The design employs a massively parallel processing unit comprised of 32 Processing Elements (PEs). These PEs operate concurrently, receiving weights from a dedicated SRAM

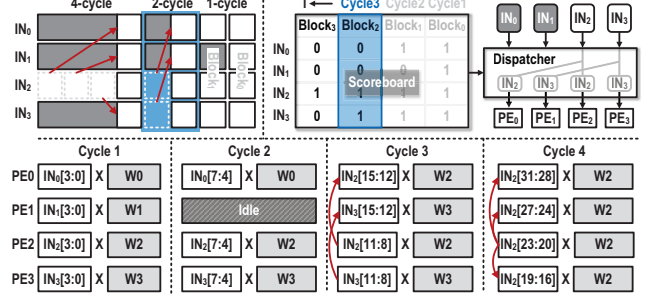


Fig. 11. Progressive sparsity balance.

buffer and input activations from a specialized dispatcher. Each PE performs 4-bit arithmetic logic unit (ALU) operations on its assigned data. The resulting partial sums are concatenated within subsequent adder and temporary buffer stages for further processing. The system performance is evaluated to quantify the throughput improvement to the overhead introduced by the PSB. This evaluation is normalized to a baseline value of 1 when using only two parallel PEs. The results indicate that 8 PEs is the optimal configuration, yielding a $1.37\times$ performance compared to the baseline scenario.

To summarize, the pre-processing unit generates a sparser bit stream. A progressive sparsity detection mechanism skips 0 induced by the data difference. A progressive sparsity balance assists in balancing the workload between PEs.

VI. SIMILARITY BALANCE

While the progressive sparsity balance technique effectively mitigates workload imbalance among PEs, a more fundamental challenge remains: intrinsic imbalance at the DRAM bank level. Specifically, the hybrid bonding DRAM bank operates independently, and each bank stores a tiling feature map. Banks containing monotonous background regions exhibit higher data similarity, leading to more opportunities for skipping redundant computations during the processing phase. As shown in Fig. 12, the consistent color across Banks 0, 2, and 3 indicates similar data. After progressive sparsity balance, most of the redundant computing is skipped by the PE units. In contrast, the bank containing the region of interest (ROI) (bank 1) typically exhibits higher data complexity and less inherent similarity, ultimately dominating the system latency.

A. Direct Workload Balance Overhead

To balance the bank-level workload, the straightforward solution is to directly copy DRAM data from busy to free banks. However, data movement introduces significant overhead, compromising the overall efficiency of the hybrid bonding architecture with respect to bandwidth and power consumption. A remote access to other banks on the Network-on-Chip (NoC), is both time-consuming (71.9 ms per remote access) and power-intensive (9.37×10^6 pJ per remote access) according to prior power/latency models [33], [34].

Bandwidth: In hybrid bonding architecture, each logic bank relies on the Network-on-Chip (NoC) for inter-bank communication. When multiple workloads are collocated on the

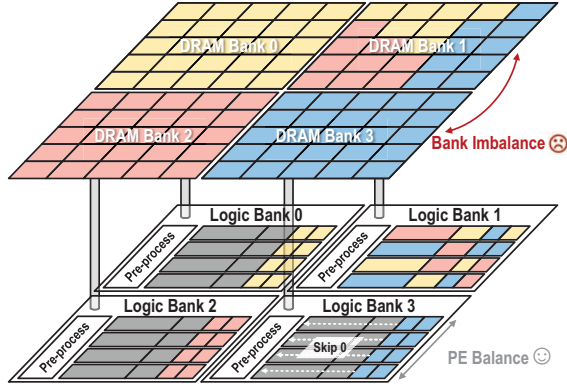


Fig. 12. The illustration for bank-level imbalance.

accelerator, groups of logic banks operate under a producer-consumer model: one logic bank concludes its pre-processing stage and transmits the resulting intermediate results through the NoC to a neighboring bank for subsequent processing steps. The NoC bandwidth is designed to match the PE unit throughput. This ensures minimal outstanding data movement requests and avoids frequent back pressure, also reduces the NoC overhead. To this end, if the workload balance task also requires moving DRAM data through NoC, DRAM data and intermediate data would compete for the NoC bandwidth.

Power: Because of parasitic capacitance and resistor, data movement from DRAM I/O consumes significant power consumption, at least 1.17 pJ/b [35]. Previous work indicates that transferring a bit ‘1’ over the wire forms a path from VDD to GND, leading to 37% higher power consumption compared to a bit ‘0’ [36]. The movement of workload would result in high power consumption due to the lengthy wire path. Additionally, assuming bits 0/1 are randomly distributed, around half of the transferring bits are energy-expensive bit ‘1’.

B. Similarity Entanglement

One opportunity is observed that data similarity also exists among remote banks. With remote similarity, the moved workload can also be converted to a sparser difference and further compressed, which relieves the burden of NoC and reduces the number of bit ‘1’.

1) *Remote Similarity:* This work underscores the existence of inter-bank similarity in two key aspects: First, when a continuous task is partitioned and distributed across separate DRAM banks, pixels bordering these divisions often exhibit similarity. Second, certain computations, like the attention mechanism in vision transformers, involve computing the relevance between distant information in different image patches. Research has demonstrated the presence of global similarity across these patches [6]. This similarity translates to redundant computation, as evidenced by the finding that 46.1% of data in distinct patches exhibits similarity, leading to 35.1% redundant computations on the ImageNet dataset. Therefore, it is necessary to exploit the remote similarity, to realize a more efficient workload balance.

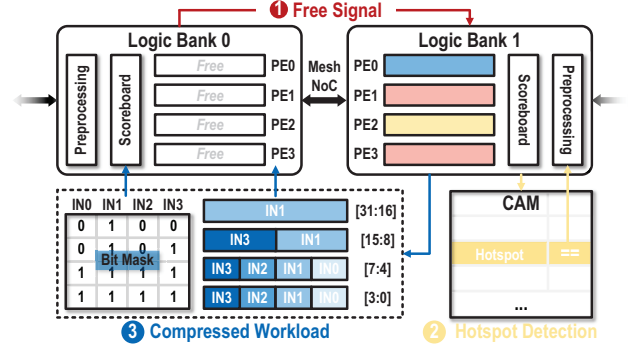


Fig. 13. The illustration of similarity entanglement detection.

2) *Similarity Entanglement Balance (SEB):* Remote similarity provides a potential opportunity to optimize bank-level balance, i.e. Instead of transferring entire data blocks, only the difference (XOR) between the hotspot and subsequent data is transmitted across the NoC, which is expected to be much sparser and can be compressed. The scoreboard detects the all-0 blocks, which are skipped during transfer to relieve the NoC bandwidth pressure. And the remaining non-all-0 block also benefits from lower transferring power because of more ‘0’ occurring on the data path. The similarity entanglement among remote banks is detected based on the proposed CAM architecture. The detailed workflow is provided in Fig. 13.

Step 1 Upon completing its assigned workload, a logic bank identifies neighboring banks with a free signal using dedicated hardware circuitry. This signal indicates the availability of resources in the neighboring bank.

Step 2 Next, the busy bank issues a content search request to the CAM unit, which compares the data to be processed and existing hotspots within CAM. The comparison is similar to the process introduced in Sec. IV-B. As the CAM is shared by 4 banks, the search key is also likely to match the hotspot from the remote bank because of remote similarity.

Step 3 The matched hotspot is designated as the reference to perform XOR with the following read-out data from the DRAM bank. The differential results from near-memory pre-processing units are compressed by skipping the all-0 data block and the bit mask denotes the non-zero data block. Data transmission initiates with the least significant 4-bit block. The non-zero data block is densely compressed to avoid wasting NoC bandwidth. The free logic bank begins to compute once it receives the data packet. The bit mask can be directly written into the scoreboard to achieve inter-PE balance within the free logic bank, as explained in Section V-B.

To summarize, remote similarity is exploited to compress the workload during bank-level balance, all-0 data blocks are skipped during movement, while other blocks also benefit from reduced movement power because of more bit 0.

VII. PUTTING IT ALL TOGETHER

Our approach exploits the bank-level parallelism of 3D DRAM and incorporates a CAM-based similarity detection and sparse computing unit to capitalize on data similarity

opportunities in parallel. Upon receiving the first burst from the DRAM bank, the CAM is consulted to identify matched ‘hotspot’. Subsequent accesses are compared with the hotspot data in a pre-processing unit, generating a ‘sparse difference’. A scoreboard then analyzes the difference results and identifies redundant 0 blocks. The scoreboard issues valid block numbers to a dispatcher, which distributes balanced workloads among parallel PEs. Once each logic bank completes its assigned workload, the remote data similarities are leveraged to further compress the workload before transmitting it through the network-on-chip (NoC). This compressed data facilitates bank-level workload balancing, optimizing resource utilization.

VIII. IMPLEMENTATION AND METHODOLOGY

Hardware Simulation: We developed a simulation framework for the hybrid bonding accelerator by augmenting an open-source trace-driven simulator, DRAMSim 3.0 [37], and builds upon established methodologies similar to prior near-memory computing projects like GradPIM. [38]. The simulator is adapted to incorporate all the proposed design improvements, enabling it to generate detailed timing and power consumption data for the entire DRAM system. The original DRAM-related library is modified by a 17nm hybrid bonding DRAM design provided by the collaborator, including timing/power lib and instruction protocol. The digital unit of the logic die is implemented by Verilog HDL and the CAM is verified by building the circuit schematic. Then the design of the logic die is synthesized and layout with TSMC 12nm CMOS technology. The specification is shown in Table. I. The logic die is composed of 16 cores and each core contains 4×4 logic banks built in a mesh topology. The number of logic banks exactly matches with DRAM bank parallelism. Each logic bank achieves 0.96 TFlops with 32 PE, which is decided based on the experiment and will be discussed in Sec. IX-A. The DRAM die is composed of 256 DRAM banks, each of which owns 32 MB capacity and operates at 500 MHz.

Benchmark: To comprehensively evaluate the performance of the proposed system, we employ a diverse set of benchmark workloads and datasets, like ResNet-50, YOLO-V5, 3D UNet from the MLPerf [39] for image-recognized tasks; Stable Diffusion-V1 [7] (S-Diffusion) for image-generated tasks; LSS for BEV Point Cloud Semantic Segmentation [40]. Emerging attention-based models are also incorporated, such as ViT-Base (ViT-B) and ViT-Huge (ViT-H) [6]. The inference results of this work are tested based on the image-based datasets, including CoCo2017 [41] / ImageNet [20] (Image-based) and autonomous datasets including KITTI [42] and nuScenes [43].

IX. EVALUATION RESULTS

A. Hardware Design Space

This work explores the design space of the hybrid bonding architecture, focusing on three key components: computing unit, on-chip buffer, and similarity-related hardware. Hardware configurations of these components are carefully fine-tuned based on the experiment results. This optimization aims to establish a competitive hybrid bonding baseline.

TABLE I
DESIGN SPECIFICATION OF DRAM AND LOGIC DIE.

Logic Die	Specification
Area	25mm × 28mm (12nm technology)
Core	16 cores 4×4 logic banks/core 32 PE/logic bank 0.96TFlops/logic bank
SRAM Buffer	112KB SRAM/logic bank
CAM	12 KB/4 DRAM Banks
NoC	Mesh topology, 128-bit flit width
Frequency	1GHz
3D DRAM Die	Specification
Area	25mm × 28mm (17nm Face-to-Face DRAM Bonding technology)
Array capacity	8GB, 256 Banks, 32MB/bank (w/ ECC 64 correct 1)
Data rate	64 I/O per 2MB, 500MHz x64 burst 16.38TB/s Bandwidth/Chip
Timing constraint	tRCD = 16.4ns, tCL = 1.7ns, tRAS = 33.6ns, tRP = 11.7ns, tRC = 45.3ns
Refresh Period tREF	<85C tREF = 32 ms <95C tREF = 16 ms <105C tREF = 8 ms <115C tREF = 4 ms
Operating voltage	VDD(main voltage) 1.2 ± 0.02V VCC(circuit voltage) 1.1 ± 0.06V Vios (IO switch voltage) 0.9 ± 0.05V

1) *PE and SRAM:* As each logic bank directly corresponds to a DRAM bank, the total area budget of a logic bank remains fixed. Consequently, SRAM and PEs compete for the limited available area resource, as the Eq. (2) demonstrates.

$$Area = \alpha \times PE_{ops} + \beta \times SRAM_{MB}, \quad (2)$$

where α ($0.953 \sim 1.102\text{mm}^2/\text{TOPs}$) denotes the operation density, and β ($2.255 \sim 2.618\text{mm}^2/\text{MB}$) represents the storage density. The α and β parameters are acquired based on the synthesis result, which varies due to routing.

The proposed hybrid bonding architecture offers significantly increased bandwidth for logic units. Empirically, the on-chip buffer size should be shrunk while expanding the processing capabilities of the PEs. Fig. 14 illustrates the impact of reduced SRAM sizes on throughput performance for diverse AI models, which regards the 192 KB SRAM as the baseline. As revealed by Fig. 14, the optimal SRAM buffer sizes for attention models Vision Transformer-base (ViT-B) and Vision Transformer-huge (ViT-H) are 192 KB, and 179.2 KB respectively. In comparison, the convolution-dominated networks, like ResNet-50 and YOLO-v5, achieve the best throughput with smaller SRAM sizes of 115.2 KB and 76.8 KB, respectively. The reason is that the attention-based models require larger buffers to store weights and the QKV matrices, while the convolution networks contain fewer parameters. To achieve balanced performance across diverse models, the average performance is employed to determine the optimal SRAM size, which is 112 KB and the corresponding PE throughput could be acquired by Eq. 2 as 0.96 TFlops.

2) *CAM:* Increasing the CAM size improves the hit rate for identifying matching hotspots, enabling the exploitation of more similarity opportunities to accelerate computation.

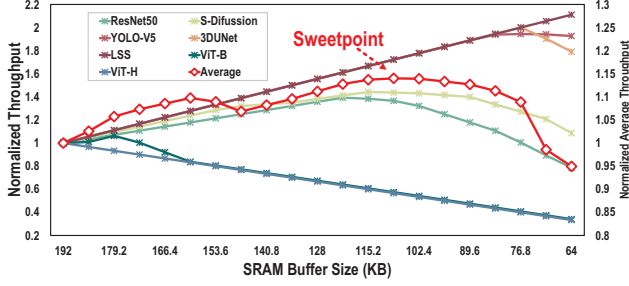


Fig. 14. Design space exploration between PE and SRAM.

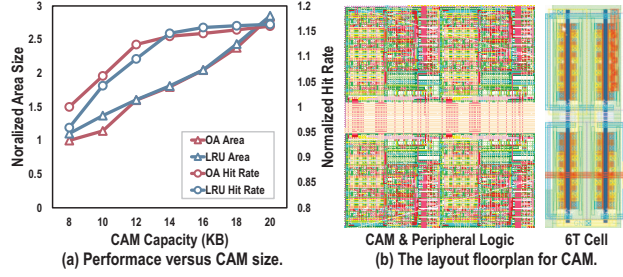


Fig. 15. CAM Design and Layout

However, this expansion also incurs a non-negligible area overhead, even with the use of compact 6T cells. According to experiment results in Fig. 15(a), a capacity of 12 KB is identified as the optimal choice, representing the ridge point of the hit rate curve. The exploration of more sophisticated eviction policies, such as Least Recently Used (LRU), yielded no significant improvement in hit rate (even resulting in a slight 0.77% decrease compared to OA). Additionally, this approach introduced an additional 4.2% area overhead. To verify the functionality and area efficiency of the chosen design, a circuit schematic and layout are implemented, as shown in Fig. 15(b). The 6T cell floorplan is inspired by previous work [44]. The area efficiency reaches 2.95Mb/mm², $1.64\times \sim 2.03\times$ higher than prior CAMs [28], [31], [32].

3) *Scoreboard*: While a larger scoreboard allows for wider workload distribution, it also incurs a significant area overhead. Fig. 16(a) illustrates the trend of normalized throughput versus different scoreboard sizes. Compared with the 2-entry baseline, the 32-entry scoreboard accelerates $1.93\times$ for ResNet-50, $2.35\times$ for YOLO-v5, and $1.87\times$ for ViT-B. However, the area overhead grows exponentially with the size of entries as shown in Fig. 16(b). The performance of throughput over area overhead illustrates that the 8-entry scoreboard is the optimal choice.

4) *NoC*: The logic banks are interconnected through a 2D-mesh topology, responsible for exchanging intermediate results, also transferring DRAM data to balance the workload. Fig. 17(a) shows the NoC utilization for different models. It is observed that the attention-based models acquire higher bandwidth compared to the convolution-dominated networks. The ViT-H requires up to 137.2Gb/s NoC bandwidth, while the ResNet-50 merely requires 6.44 Gb/s bandwidth. With remote

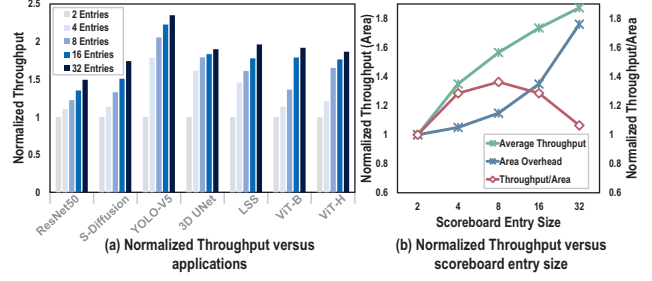


Fig. 16. Scoreboard entry size effect on throughput and area.

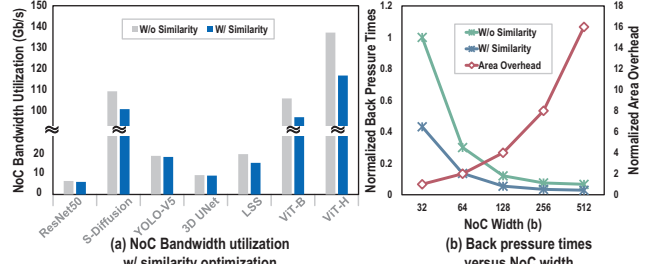


Fig. 17. NoC bandwidth requirement.

similarity compression, the bandwidth requirements are further reduced. For instance, ViT-B initially needs 106.1 Gb/s NoC bandwidth, which is optimized to 97.5 Gb/s after compressing the workload by skipping all-0 blocks.

The utilization of NoC bandwidth is estimated based on long-term data movement. However, the data movement through NoC may occur in high bursts within a short period, leading to NoC congestion. Quantifying this congestion is difficult, hence this work employs back pressure signals as an indicator. Fig. 17(b) shows the trend of back pressure times versus the NoC bit width. Increasing the bit width from 32 bits to 128 bits significantly reduces back pressure occurrences by $8.6\times$. On the other hand, increasing NoC width also results in a linear increase in area overhead. Therefore, the width of NoC is designed to be 128-b between logic banks.

B. Similarity Analysis

This work leverages a crucial observation: data similarity within memory banks. This similarity is exploited to generate sparse representations, enabling computation skipping or compressed data transfer. We conducted experiments to evaluate the generated sparsity after applying similarity processing. Sparsity is measured as the ratio of zeros within each data element. Fig. 18 illustrates the sparsity ratio across different workloads for both local (intra-bank) and remote (inter-bank) similarity. As expected based on the CSE principle, local similarity achieves a $1.354\times$ sparsity ratio improvement, which is 27% higher than remote similarity.

Fig. 19(a) further demonstrates the effectiveness of local similarity detection in improving sparsity. On average, sparsity is increased by 29.3%, with 41.8% of logic banks achieving an improvement exceeding 20%. Similarly, Fig. 19(b) showcases the sparsity enhancements obtained through remote similarity

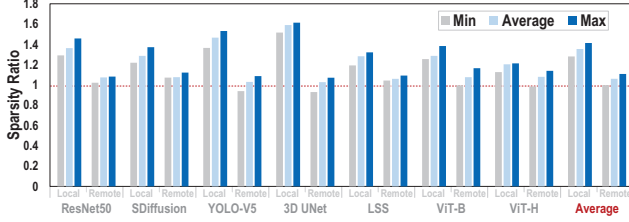


Fig. 18. The sparse ratio with local similarity and remote similarity.

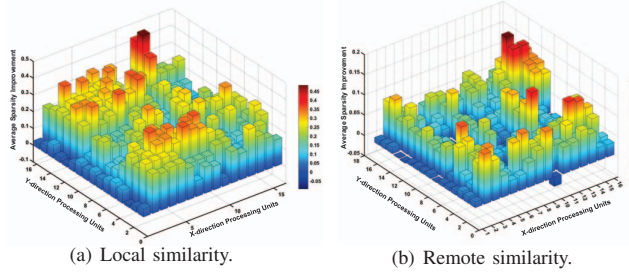


Fig. 19. The statistical results for local and remote similarity.

detection. Here, logic banks exhibit an average sparsity improvement of 16.7%, with 19.1% exceeding a 20% increase.

C. Overall Performance

We compare our work against state-of-the-art (SOTA) AI accelerators Tenstorrent Wormhole and Google TPUv3 [45]–[47], due to similar fabrication processes. The Wormhole is a representative 2D tiled AI accelerator and has 771 mm² die area, which realizes 110 TFLOPs and includes up to 120 MB on-chip SRAM. The off-chip DRAM is 12 GB GDDR and achieves 384 GB/s DRAM Bandwidth. And we also test the result when replacing the GDDR DRAM with an HBM3 DRAM (896 GB/s). The TPU is a well-known AI accelerator realizing 123 TFLOPs with 32 MB on-chip SRAM and 900 GB/s HBM. The performance of these industry products is evaluated using an analytical model with known design parameters [48].

To objectively evaluate the performance, we normalize all chips' areas and scale corresponding specifications. The benefits of this work are profiled in 2-step. First, following the design considerations outlined in Sec. II and accounting for the associated area overhead, we integrate hybrid bonding DRAM within the designated chip area. We further optimized hardware parameters after design space exploration (Sec. IX-A). This design significantly enhances memory bandwidth and serves as the 3D baseline. Second, building upon the 3D baseline, we leverage data similarity to accelerate computing. 'BaseSim' utilizes local data similarity for computation acceleration, 'BaseSimPsb' incorporates progressive sparse balance for optimized workload distribution among PEs, and 'BaseSimPsbRE' further improves performance by incorporating both local and remote similarity mechanisms. The performance of each design is evaluated by deploying different AI models.

The throughput performance is shown in Fig. 20(a). Compared to the Wormhole + DDR and HBM, the 3D baseline achieves 6.72 $\times$ and 2.73 $\times$ throughput improvement, respec-

tively. Against the Google TPUv3, the 3D baseline delivers 2.34 $\times$ throughput. Two key factors contribute to these impressive gains. First, the bandwidth of 3D memory is an order of magnitude higher than that of 2D/2.5D DRAM. Second, the high bandwidth allows allocating larger area to PE units.

Compared with the 3D baseline, this work explores the data similarity for further performance improvements. The initial local similarity detection enhances the computing throughput by 1.21 $\times$. Further, the progressive sparse balance scheme, which distributes the workload among PE units, yields 1.35 $\times$ throughput improvement. Finally, integrating remote similarity mechanisms to achieve bank-level workload balance further enhances performance by 1.43 $\times$ on average. With three techniques working together, the ResNet-50 realizes 1.46 $\times$ acceleration, and ViT-B improves throughput by 1.38 $\times$. The autonomous task LSS outperforms the baseline by 1.35 $\times$.

Fig. 20(b) provides the normalized power consumption for different models. For a more intuitive demonstration, the Y-axis is set to the reciprocal of the normalized power consumption. As observed, compared with the 2D/2.5D DRAM, hybrid bonding baseline reduces memory access power by shortening the data path, thus achieving 1.48 $\times$ ~ 2.50 $\times$ power reduction. Compared with the 3D baseline, the similarity detection benefits power consumption by skipping redundant computing, which brings the 1.13 $\times$ power consumption reduction. Moreover, the progressive sparsity balance scheme further reduces 1.26 $\times$ power on average. Remote similarity generates more bit 0 during workload balance, which accomplishes an average of 1.37 $\times$ power consumption reduction over the 3D baseline.

Fig. 20(c) presents the energy efficiency performance. The hybrid bonding integration improves the bandwidth with lower power consumption. The baseline achieves 3.49 $\times$ and 2.89 $\times$ improvement over Wormhole and TPU. Leveraging data similarity further optimizes the 3D baseline, resulting in a 1.97 $\times$ improvement in energy efficiency. By stacking two wafers together, the hybrid bonding scheme naturally achieves 2.67 $\times$ ~7.68 $\times$ area efficiency over 2D/2.5D integration. The similarity optimization further improves the area efficiency by 1.43 $\times$ over the 3D baseline, as shown in Fig. 20(d). Together, the architecture demonstrates an average energy efficiency improvement of 5.69 $\times$ ~28.13 $\times$ and area efficiency enhancement of 3.82 $\times$ ~10.98 $\times$ over SOTA accelerators.

The logic die is implemented with TSMC 12nm CMOS technology. The area of each logic bank is exactly matching with the DRAM bank. Fig. 21 shows the floorplan of the logic bank, where the PE array (37%) and SRAM buffer (26%) are the primary components. The DRAM I/O and the memory controller occupy 18% area. The other peripheral has a 7% area. The CAM area reaches 0.0317 mm² and is shared by 4 logic banks. The similarity-related hardware (including near-memory pre-processing unit, scoreboard, dispatcher) incurs less than 1% area overhead.

X. RELATED WORKS

Several existing works have explored mechanisms to enhance activation data sparsity for improved processing effi-

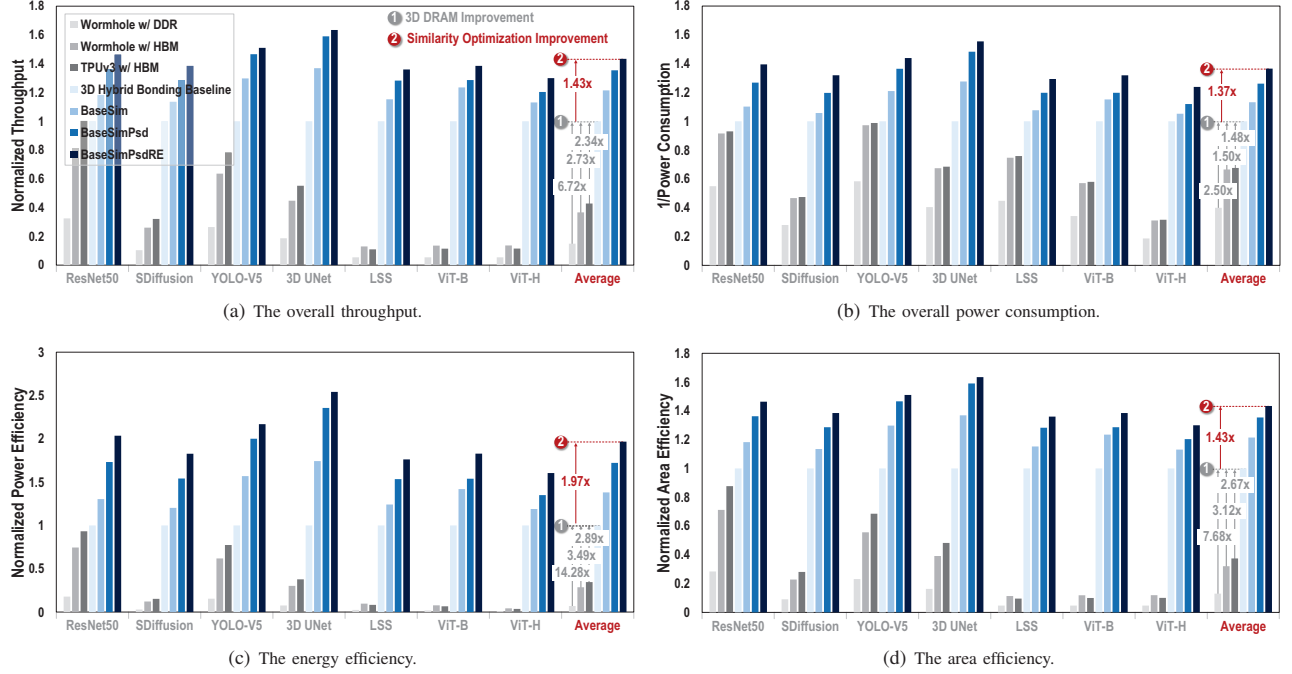


Fig. 20. The overall hardware performance.

ciency. These approaches can be categorized into software and hardware techniques. The software techniques are orthogonal to this work, as our work is hardware-only. For hardware techniques, Some techniques aim to selectively compute only the significant activations, such as selecting top-k activations for computation and discarding smaller values [49]–[51]. While these methods can improve sparsity, they may degrade accuracy. Other hardware-based approaches focus on compressing activation data through novel encoding mechanisms [52]–[54]. However, these techniques often require complex decoding mechanisms, incurring significant area overhead within fine-grained 3D DRAM banks.

XI. CONCLUSION

This paper presents a novel hybrid bonding design that leverages the CSE to accelerate vision AI models. The paper offers a comprehensive analysis of the hybrid bonding design, including design considerations and integration overhead. Building on the 3D DRAM, a CSE-aware accelerator is proposed with several features. First, an area-efficient content addressable memory is employed to detect the similarity. Second, a progressive sparse detector accelerates computing based on similarity-induced sparse features. Third, the remote similarity is thoroughly analyzed to optimize inter-bank workload balance. Finally, the accelerator design space is explored by introducing 3D DRAM, including SRAM buffer, PE capability, and similarity-related hardware. Evaluated on a diverse set of benchmark workloads and datasets, this work improves the energy efficiency by an average of $5.69\times\sim 28.13\times$, and area efficiency by $3.82\times\sim 10.98\times$ over SOTA accelerators.

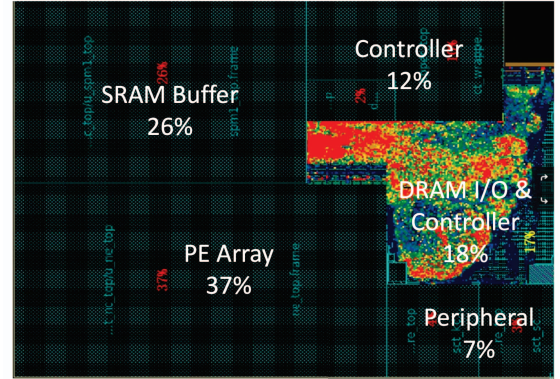


Fig. 21. Floorplan of the logic bank.

ACKNOWLEDGEMENTS

This work was supported in part by the National Key R&D Project under Grant 2023YFB4403100; in part by the NSFC under Grant 62125403, and Grant 92164301; in part by the National Science and Technology Major Project under Grant 2022ZD0115201; in part by the National Key Research and Development Program under Grant 2021ZD0114400; Beijing S&T Project Z221100007722023; in part by the Beijing National Research Center for Information Science and Technology; and in part by the Beijing Advanced Innovation Center for Integrated Circuits. Corresponding authors: Shouyi Yin (yinsy@tsinghua.edu.cn) and Yang Hu (hu_yang@tsinghua.edu.cn).

REFERENCES

- [1] R L K Venkateswarlu, G R Vani Vasantha Kumari, and Jayasri. Speech recognition by using recurrent neural networks. *international journal of scientific engineering research*, 2013.
- [2] Demetris Vrontis, Michael Christofi, Vijay Pereira, Shlomo Tarba, and Eleni Trichina. Artificial intelligence, robotics, advanced technologies and human resource management: a systematic review. *The International Journal of Human Resource Management*, (3):1–30, 2021.
- [3] Alex Krizhevsky, I. Sutskever, and G. Hinton. ImageNet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25(2), 2012.
- [4] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *Computer Science*, 2014.
- [5] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, and Andrew Rabinovich. Going deeper with convolutions. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [6] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, and S. Gelly. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- [7] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.
- [8] Peng Gu, Xinfeng Xie, Yufei Ding, Guoyang Chen, Weifeng Zhang, Dimin Niu, and Yuan Xie. ipim: Programmable in-memory image processing accelerator using near-bank architecture. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 804–817. IEEE, 2020.
- [9] Zidong Du, Shaoli Liu, Robert Fasthuber, Tianshi Chen, Paolo Ienne, Ling Li, Tao Luo, Qi Guo, Xiaobing Feng, Yunji Chen, et al. An accelerator for high efficient vision processing. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 36(2):227–240, 2016.
- [10] Varnit Mittal and Bharat Bhushan. Accelerated computer vision inference with ai on the edge. In *2020 IEEE 9th International Conference on Communication Systems and Network Technologies (CSNT)*, pages 55–60. IEEE, 2020.
- [11] Nanchini Chandramoorthy, Giuseppe Tagliavini, Kevin Irick, Antonio Pullini, Siddharth Advani, Sulaiman Al Habsi, Matthew Cotter, John Sampson, Vijaykrishnan Narayanan, and Luca Benini. Exploring architectural heterogeneity in intelligent vision systems. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, pages 1–12, 2015.
- [12] Hyoukjun Kwon, Liangzhen Lai, Michael Pellauer, Tushar Krishna, Yu-Hsin Chen, and Vikas Chandra. Heterogeneous dataflow accelerators for multi-dnn workloads. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 71–83, 2021.
- [13] Yuhao Zhu, Anand Samajdar, Matthew Mattina, and Paul Wharmouth. Euphrates: Algorithm-soc co-design for low-power mobile continuous vision. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pages 547–560, 2018.
- [14] Alejandro Nieto, David L. Vilariño, and Victor M. Brea. Precision: A reconfigurable simd/mimd coprocessor for computer vision systems-on-chip. *IEEE Transactions on Computers*, 65(8):2548–2561, 2016.
- [15] Zen3: The AMD 2nd-generation 7nm x86-64 microprocessor core. In *2022 IEEE International Solid - State Circuits Conference - (ISSCC)*, 2022.
- [16] Dimin Niu, Shuangchen Li, Yuhao Wang, Wei Han, Zhe Zhang, Yijin Guan, Tianchan Guan, Fei Sun, Fei Xue, Lide Duan, et al. 184qps/w 64mb/mm 2 3d logic-to-dram hybrid bonding with process-near-memory engine for recommendation system. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 1–3. IEEE, 2022.
- [17] Bai Fujun, Jiang Xiping, Wang Song, Yu Bing, Tan Jie, Zuo Fengguo, Wang Chunjuan, Wang Fan, Long Xiaodong, Yu Guoqing, et al. A stacked embedded dram array for lpddr4/x using hybrid bonding 3d integration with 34gb/s/1gb 0.88 pj/b logic-to-memory interface. In *2020 IEEE International Electron Devices Meeting (IEDM)*, pages 6–6. IEEE, 2020.
- [18] Kaiping He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pages 770–778. IEEE Computer Society, 2016.
- [19] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- [20] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- [21] BERNHARD BURGSTALLER and FRIEDRICH PILLICHSHAMMER. The average distance between two points. *Bulletin of the Australian Mathematical Society*, 80(3):353–359, 2009.
- [22] Urban Borštnik, Joost VandeVondele, Valéry Weber, and Jörg Hutter. Sparse matrix multiplication: The distributed block-compressed sparse row library. *Parallel Computing*, 40(5-6):47–58, 2014.
- [23] Geonhwa Jeong, Sana Damani, Abhimanyu Rajeshkumar Bambhaniya, Eric Qin, Christopher J. Hughes, Sreenivas Subramoney, Hyesoon Kim, and Tushar Krishna. VEGETA: vertically-integrated extensions for sparse/dense GEMM tile acceleration on cpus. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*, pages 259–272. IEEE, 2023.
- [24] Yifan Yang, Joel S. Emer, and Daniel Sánchez. Isosceles: Accelerating sparse cnns through inter-layer pipelining. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*, pages 598–610. IEEE, 2023.
- [25] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. Retrospective: EIE: efficient inference engine on sparse and compressed neural network. *CoRR*, abs/2306.09552, 2023.
- [26] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010.
- [27] Pavlo Molchanov, Stephen Tyree, Tero Karras, Timo Aila, and Jan Kautz. Pruning convolutional neural networks for resource efficient inference. *arXiv preprint arXiv:1611.06440*, 2016.
- [28] Yi Huang, Zhiyu Chen, Dai Li, and Kaiyuan Yang. Cama: Energy and memory efficient automata processing in content-addressable memories. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 25–37, 2022.
- [29] Cheng Xin Xue, Wei Cheng Zhao, Tzu Hsien Yang, Yi Ju Chen, and Meng Fan Chang. A 28-nm 320-kb team macro using split-controlled single-load 14t cell and triple-margin voltage sense amplifier. *IEEE Journal of Solid-State Circuits*, 54(10):2743–2753, 2019.
- [30] Kyeongho Lee, Woong Choi, and Jongsun Park. A 65-nm 0.6-fj/bit/search ternary content addressable memory using an adaptive match-line discharge. *IEEE Journal of Solid-State Circuits*, PP(99):1–1, 2020.
- [31] Helena Caminal, Kailin Yang, Srivatsa Srinivasa, Akshay Krishna Ramanathan, Khalid Al-Hawaj, Tianshu Wu, Vijaykrishnan Narayanan, Christopher Batten, and José F. Martínez. Cape: A content-addressable processing engine. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 557–569, 2021.
- [32] Jinn-Shyan Wang, Hung-Yu Li, Chia-Cheng Chen, and Chingwei Yeh. An and-type match-line scheme for energy-efficient content addressable memories. In *ISSCC. 2005 IEEE International Digest of Technical Papers. Solid-State Circuits Conference, 2005.*, pages 464–610 Vol. 1, 2005.
- [33] Gwangsun Kim, John Kim, Jung Ho Ahn, and Jaeha Kim. Memory-centric system interconnect design with hybrid memory cubes. In *Proceedings of the 22nd International Conference on Parallel Architectures and Compilation Techniques*, pages 145–155, 2013.
- [34] Oumaima Matoussi. Noc performance model for efficient network latency estimation. In *2021 Design, Automation Test in Europe Conference Exhibition (DATE)*, pages 994–999, 2021.
- [35] Mike O’Connor, Niladri Chatterjee, Donghyuk Lee, John Wilson, Aditya Agrawal, Stephen W. Keckler, and William J. Dally. Fine-grained dram: Energy-efficient dram for extreme bandwidth systems. In *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 41–54, 2017.
- [36] D. Lee, M. O’Connor, and N. Chatterjee. Reducing data transfer energy by exploiting similarity within a data transaction. In *IEEE International Symposium on High Performance Computer Architecture*, 2018.

- [37] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob. DRAMsim3: a cycle-accurate, thermal-capable DRAM simulator. *IEEE Computer Architecture Letters*, PP(99):1–1, 2020.
- [38] H. Kim, H. Park, T. Kim, K. Cho, and J. Lee. Gradpim: A practical processing-in-DRAM architecture for gradient descent. 2021.
- [39] P. Mattson, C. Cheng, C. Coleman, G. Diamos, P. Micikevicius, D. Patterson, H. Tang, G. Y. Wei, P. Bailis, and V. Bittorf. Mlperf training benchmark, 2019.
- [40] Jonah Philion and Sanja Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3D. 2020.
- [41] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014.
- [42] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3354–3361. IEEE, 2012.
- [43] Holger Caesar, Varun Bankiti, Alex H Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuscenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11621–11631, 2020.
- [44] Tyj Chang, Y. H. Chen, W. M. Chan, H. Cheng, P. S. Wang, Y. Lin, H. Fujiwara, R. Lee, H. J. Liao, and P. W. Wang. A 5-nm 135-mb sram in euv and high-mobility channel FinFET technology with metal coupling and charge-sharing write-assist circuitry schemes for high-density and Low-VMIN applications. *IEEE Journal of Solid-State Circuits*, (1):56, 2021.
- [45] Drago Ignjatović, Daniel W. Bailey, and Ljubisa Bajić. The wormhole ai training processor. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 356–358, 2022.
- [46] Thomas Norrie, Nishant Patil, Doe Hyun Yoon, George Kurian, and David Patterson. The design process for google’s training chips: Tpuv2 and tpuv3. *IEEE Micro*, PP(99):1–1, 2021.
- [47] Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David Patterson. Ten lessons from three generations shaped google’s tpuv4i : Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–14, 2021.
- [48] Sameer Kumar, Victor Bitorff, Dehao Chen, Chiachen Chou, Blake Hechtman, HyoukJoong Lee, Naveen Kumar, Peter Mattson, Shibo Wang, Tao Wang, Yuanzhong Xu, and Zongwei Zhou. Scale mlperf-0.6 models on google tpu-v3 pods, 2019.
- [49] Zhi Gang Liu, Paul N. Whatmough, Yuhao Zhu, and Matthew Mattina. S2TA: exploiting structured sparsity for energy-efficient mobile CNN acceleration. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2022, Seoul, South Korea, April 2-6, 2022*, pages 573–586. IEEE, 2022.
- [50] Weizhe Hua, Yuan Zhou, Christopher De Sa, Zhiru Zhang, and G. Edward Suh. Boosting the performance of CNN accelerators with dynamic fine-grained channel gating. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2019, Columbus, OH, USA, October 12-16, 2019*, pages 139–150. ACM, 2019.
- [51] Liu Liu, Zheng Qu, Lei Deng, Fengbin Tu, Shuangchen Li, Xing Hu, Zhenyu Gu, Yufei Ding, and Yuan Xie. DUET: boosting deep neural network efficiency on dual-module architecture. In *53rd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2020, Athens, Greece, October 17-21, 2020*, pages 738–750. IEEE, 2020.
- [52] Jie-Fang Zhang, Ching-En Lee, Chester Liu, Yakun Sophia Shao, Stephen W. Keckler, and Zhengya Zhang. SNAP: an efficient sparse neural acceleration processor for unstructured sparse deep neural network inference. *IEEE J. Solid State Circuits*, 56(2):636–647, 2021.
- [53] Tanner Andrusis, Joel S. Emer, and Vivienne Sze. RAELLA: reforming the arithmetic for efficient, low-resolution, and low-loss analog PIM: no retraining required! In Yan Solihin and Mark A. Heinrich, editors, *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*, pages 27:1–27:16. ACM, 2023.
- [54] Bitar Darvish Rouhani, Ritchie Zhao, Venmugil Elango, Rasoul Shafipour, Mathew Hall, Maral Mesmakhosroshahi, Ankit More, Levi Melnick, Maximilian Golub, Girish Varatkar, Lai Shao, Gaurav Kolhe, Dmitry Melts, Jasmine Klar, Renee L’Heureux, Matt Perry, Doug Burger, Eric S. Chung, Zhaoxia (Summer) Deng, Sam Naghshineh, Jongsoo Park, and Maxim Naumov. With shared microexponents, A little shifting goes a long way. In Yan Solihin and Mark A. Heinrich, editors, *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*, pages 83:1–83:13. ACM, 2023.